

EE272 Homework 6

Synthesis

Due: Feb 19, 2026, 11:59 pm

Released: Feb 12, 2026

1 Introduction

This assignment focuses on synthesizing your accelerator RTL design into a gate-level netlist using Synopsys Design Compiler. The goal of this assignment is to apply what you have learned about synthesis and static timing analysis tools to quantify your design quality in terms of area and cycle time and to then iterate on improving these metrics.

First of all, make sure you are in the Singularity environment.

```
$ /farmshare/home/classes/ee/admin/software/bin/centos7.sh
```

To get started, first clone the homework repository:

```
$ git clone https://code.stanford.edu/ee272/dnn-accelerator-syn.git
```

Then make sure you are at the top level of this code repo before running any commands:

```
$ cd dnn-accelerator-syn
```

Next, set up the environment variables by sourcing the setup file. This setup is required for every new shell you open.

```
$ source setup.csh
```

The remainder of this handout will describe the various tools and libraries you will be using in this homework which include: mflowgen, a flexible flow generator; SkyWater PDK, an open-source 130nm PDK; and OpenRAM, an open-source memory compiler. We will then introduce two pipecleaner designs that you can use to familiarize yourself with the ASIC toolflow before introducing your task for this homework.

2 Flow Generators – mflowgen

The mflowgen tool is a modular flow specification and build-system generator for ASIC and FPGA design-space exploration built around sandboxed and modular steps. You can clone and install the tool following the instruction below. There is also a quick start guide (<https://mflowgen.readthedocs.io/en/latest/quick-start.html>), but it's outdated and not tailored to our class. The source repository is here: <https://github.com/mflowgen/mflowgen>. Please enter the hw6 repository and follow the steps below to install mflowgen. **You only need to go through these steps one time.** After that, whenever you open a new shell, make sure to be in the venv to have all the proper python packages installed (source venv/bin/activate.csh.) Make sure to source setup.csh as well.

```
# this command creates a python virtual environment
$ python3.7 -m venv venv
# enter the virtual environment by sourcing this activate script
# this step needs to be done for every new shell you open
$ source venv/bin/activate.csh
# in the virtual environment, install any python package you want
$ pip install numpy
$ git clone https://github.com/mflowgen/mflowgen.git
$ cd mflowgen
# also install mflowgen into your virtual environment
$ pip install -e .
$ deactivate; source ../venv/bin/activate.csh # to get the updated PATH
```

mflowgen allows you to programmatically define and parameterize a graph of steps (i.e., sandboxes that run anything you like) with well-defined inputs and outputs, where the graph executes some higher-level function (e.g., pushes your design through the ASIC tools). These toolflows are complex and their scripts are difficult to reuse. The primary goal of mflowgen is to modularize this toolflow into smaller steps which can be reused across projects and across technology nodes. In future homework, your graph will eventually look something like this (hard to read, zoom in if you are curious; you will be able to generate this graph yourself in the next homework):

- `synopsys-vcs-sim` – Node that runs the test using Synopsys VCS.
- `synopsys-vcd2saif-convert` – Converts the VCD file generated by running the test into a switching activity file (saif) used for power estimation.

3 SkyWater Open Source PDK

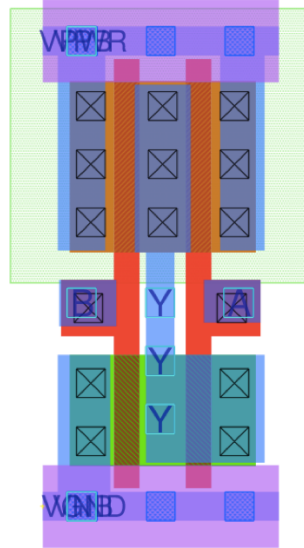
The SkyWater Open Source PDK is an open-source process design kit (PDK) that allows researchers and students to experiment with designing in a technology node without signing restrictive non-disclosure agreements or paying for licenses. Using this PDK, we can create both commercial and open-source flows to design analog and digital circuits.

However, because this is a recent effort, all tools are not supported yet. For example, in many design flows you would use Mentor Graphics Calibre tool for DRC (design-rule check) and LVS (layout-vs-schematic) checks. Running these checks requires rule decks that are not available in the open-source version of this PDK. So, for now our flow will use open-source tools (magic and netgen) for DRC and LVS. Similarly, Cadence Virtuoso is not fully supported yet, so we will use a combination of magic, ngspice and netgen for your analog design homework. Fortunately, Synopsys Design Compiler and Cadence Innovus work with this PDK, so for digital design, your experience will be similar to one you would have in the industry.

You can find more details about SkyWater Open Source PDK in its repository: <https://github.com/google/skywater-pdk>. The documentation is here: <https://skywater-pdk.readthedocs.io/en/main/>. **We have already installed this PDK for you on farmshare machines here:** </farmshare/home/classes/ee/272/PDKs/skywater-pdk/>

3.1 SkyWater Standard Cell Library

Digital synthesis tools work with libraries of pre-drawn and pre-characterized standard cells (often provided by the foundry or IP vendors like ARM). We will be using the SkyWater foundry provided (fd) high density (hd) standard cells (sc) that can be found here: /farmshare/home/classes/ee/272/skywater-pdk/libraries/sky130_fd_sc_hd/latest/cells. Note that each type (function, such as inv, nand2, etc.) of cell has its own sub-directory at the above location, inside which there exist files for several sizes (drive strengths) for that cell. **We have converted these files into the ADK format needed by mflowgen here:** </farmshare/home/classes/ee/272/skywater-130nm-adk.v2021>. For example, here is the layout for a 2 input NAND gate with size 1 (nand2_1) standard cell:



You can see the VPWR rail (blue strip at the top), the VGND rail (blue strip at the bottom), the poly (red vertical strips), contacts (cross-marked squares), input pins on metal 1 for A and B (that connect to the poly gates) and output pin Y. The physical database containing these layout shapes is known as a graphic database system (GDS) and can be found in `stdcells.gds` in the ADK: [/farmshare/home/classes/ee/272/skywater-130nm-adk.v2021/view-standard/stdcells.gds](#) (or in the corresponding folder in the PDK).

Each cell in the library is characterized with a large number of SPICE simulations and the timing information is captured in the `stdcells.lib` and `stdcells.db` files (the same information is captured in both, just different file formats). The liberty (`.lib`) file is human-readable and you can directly read it here: [/farmshare/home/classes/ee/272/skywater-130nm-adk.v2021/view-standard/stdcells.lib](#)

The library also comes with the routing technology kit (RTK) technology LEF, often called the “tech LEF”. This describes all available layers that the synthesis and place-and-route tools can use for routing. This file is also human-readable. You can read it here: [/farmshare/home/classes/ee/272/skywater-130nm-adk.v2021/view-standard/rtk-tech.lef](#). For example, the metal 1 (`met1`) parameters look like this:

```
LAYER met1
  TYPE ROUTING ;
  DIRECTION HORIZONTAL ;

  PITCH 0.34 ;
  OFFSET 0.17 ;

  WIDTH 0.14 ;                # Met1 1
  # SPACING 0.14 ;            # Met1 2
  # SPACING 0.28 RANGE 3.001 100 ; # Met1 3b
  SPACINGTABLE
```

```

    PARALLELRUNLENGTH 0
    WIDTH 0 0.14
    WIDTH 3 0.28 ;
    AREA 0.083 ;                      # Met1 6
    THICKNESS 0.35 ;

    ANTENNAMODEL OXIDE1 ;
    ANTENNADIFFSIDEAREARATIO PWL ((0 400) (0.0125 400) (0.0225 2609) (22.5 11600)) ;

    EDGECAPACITANCE 40.567E-6 ;
    CAPACITANCE CPERSQDIST 25.7784E-6 ;
    DCCURRENTDENSITY AVERAGE 2.8 ; # mA/um Iavg_max at Tj = 90oC
    ACCURRENTDENSITY RMS 6.1 ; # mA/um Irms_max at Tj = 90oC
    MAXIMUMDENSITY 70 ;
    DENSITYCHECKWINDOW 700 700 ;
    DENSITYCHECKSTEP 70 ;

    RESISTANCE RPERSQ 0.125 ;
END met1

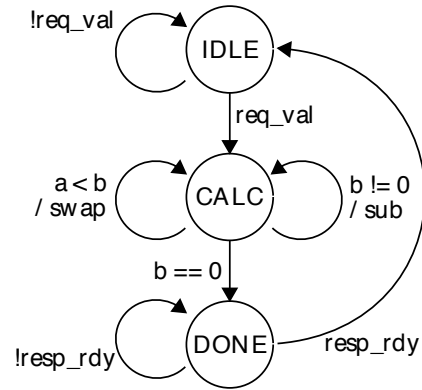
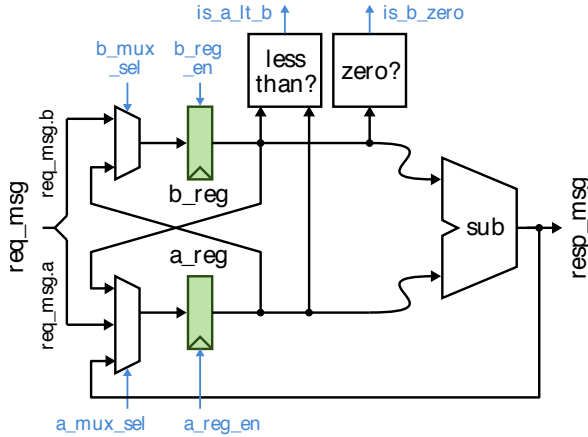
```

4 Pipe Cleaners

Pipe cleaners are small designs that run through the flow quickly and help to identify errors early. It is good practice to frequently run pipe cleaners while developing the flow before running your full design.

4.1 Greatest Common Divisor

The GcdUnit module computes the greatest common divisor function and is a great design that has only 100–200 gates to quickly spin through the flow. Importantly, it contains only standard cells and no memory macros, greatly decreasing the complexity of the design. The datapath includes two state registers and the required muxing and arithmetic units to iteratively implement Euclid’s algorithm. The diagram below shows a hybrid Moore/Mealy FSM for controlling the datapath. Mealy transitions in the calc state determine whether to swap or subtract.



Christopher Batten, Cornell University, ECE 4750 Computer Architecture

To start the pipe cleaner, first set up the environment following steps in [section 1](#) and [2](#).

Then go into the **build** directory (which you create) and configure for this design:

```
$ cd GcdUnit/build
$ mflowgen run --design ../design
$ make list
```

Then open the mflowgen graph to see what this pipe cleaner flow looks like:

```
$ make graph
```

Open the generated graph.pdf in a PDF viewer (evince in bash and xdg-open in centos).

```
$ xdg-open graph.pdf
```

You will see that the GcdUnit RTL, a constraints file, and the technology interface from skywater-130nm-adk pass into the **synopsys-dc-synthesis** node. Compare this graph to the Python description of the graph in `GcdUnit/design/construct.py`.

Run synthesis in the build directory like this:

```
$ make list
(check which step synthesis is.. we will assume it is step 4)
$ make 4
```

Then navigate into the synthesis sandbox (i.e., “4-synopsys-dc-synthesis”) and find the following files in the reports directory:

- GcdUnit.mapped.area.rpt
- GcdUnit.mapped.power.rpt

- GcdUnit.mapped.qor.rpt
- GcdUnit.mapped.timing.setup.rpt
- GcdUnit.premapped.checkdesign.rpt

The quality-of-results (QoR) report is the most useful and summarizes both timing and area. The area report is a hierarchical breakdown of the module based on the area numbers reported for each gate (from the stdcells.lib). The power report is a *high-level estimate* of the power of your design assuming about 10% switching activity (if you don't use a switching activity file generated from simulating your design) on all nets (not very accurate, but a good first-order estimate). The setup timing report contains the full trace for the most critical timing paths, with the critical path listed first.

4.1.1 mflowgen Hints

Any time you change `construct.py` or anything in your `design` folder you should rerun configuration and do a `make status` to see if step numbers changed:

```
$ mflowgen run --design ../design
$ make status
```

If you want to delete a step (for example step 4) you can do:

```
$ make clean-4
```

You do not need to delete steps if you didn't change them. For example, pretend you are iterating on step 10. You do not need to delete 1-9 over and over. Just configure on each design change and delete 10. Unless you change the inputs for that part of the graph, you don't need to rerun anything unnecessarily.

```
$ make clean-10
$ mflowgen run --design ../design
$ make status
```

Finally, you may find it tedious to edit in the design folder and configure for each small change. You do have the option to edit in your `build/[step]` folder and then run to see if small changes work by executing `./mflowgen-run` inside it. **NOTE:** You **MUST** copy these changes back to the design folder or you will lose them when you do `make clean`. Here is an example of what you could do:


```

$ cd 3-rtl
$ vim design.v
$ ./mflowgen-run
$ // repeat step 2 and 3 until satisfied
$ cp design.v ../design/rtl/design.v
$ // copy everything to design folder
$ cd ..
$ mflowgen run --design ../design
$ make status

```

4.2 Working with SRAMs

The SramUnit consists of a few gates and an SRAM generated by a memory compiler called [OpenRAM](#). The mflowgen graph for this design consists of an extra step, called “sram”, which runs OpenRAM to produce the memory specified by a config file, and writes the output files to the outputs folder. OpenRAM takes a very long time to run (~ days) especially if it is asked to run parasitics extraction to calculate accurate delays, so in this homework, you can simply use pre-generated SRAMs that can be found here: [/farmshare/home/classes/ee/272/sky130_sram_macros](#).

First open the mflowgen graph specified in `SramUnit/design/construct.py`. One way to create edges between nodes is by using the `connect_by_name()` method. For example, the rtl step explicitly specifies (i.e., in `SramUnit/design/rtl/configure.yml`) that it outputs `design.v` and the globally installed synthesis step (in `$MFLOWGEN_HOME/steps/synopsys-dc-synthesis/configure.yml`) specifies in its `configure.yml` that it inputs `design.v`. Therefore, since this file is specified as an output of a node and the input of another in the `configure.yml`, we can create this connection with:

```
g.connect_by_name(rtl, dc)
```

However, when we generate the macros, we now have several extra files that need to be specified as inputs to several downstream nodes. Instead of manually editing the `configure.yml` of all the downstream nodes, we can dynamically add inputs in `construct.py`. For example, if the sram step outputs a file called `sram.db` that needs to be an input to the dc step, we can use this alternate way of creating edges:

```
dc.extend_inputs(['sram.db'])
g.connect_by_name(sram, dc)
```

To see this visually in the graph, you can call `make graph` as before.

Run this pipe cleaner design like this:

```
$ cd $top/SramUnit/build
$ mflowgen run --design ../design
$ make list  # check which step synthesis is
$ make 5     # assuming synthesis is step 5
```

As before, navigate into the synthesis sandbox (i.e., “5-synopsys-dc-synthesis”) and find the following files in the reports directory:

- SramUnit.mapped.area.rpt
- SramUnit.mapped.power.rpt
- SramUnit.mapped.qor.rpt
- SramUnit.mapped.timing.setup.rpt
- SramUnit.premapped.checkdesign.rpt

Notice in the area report that there is a black box category where almost all of the area is accounted for. This makes sense because the design is dominated by the macro.

5 Using Catapult with Technology-Specific Memory Macros

In the previous homework, Catapult inserted a generic behavioral SRAM. Now that we want to synthesize our design, we will need to use the memory macros for our technology. There are SRAMs in three modules: InputDoubleBuffer, WeightDoubleBuffer, and SystolicArrayCore. In order to determine the size of the SRAMs that you need to generate, open the Catapult GUI and switch to the Architecture tab. Next to the memory will be the size, written as DEPTH×WIDTH.

We suggest using the pre-generated memory macros and wrapping them in a Verilog module to achieve the depth and width that you need. **For the input and weight double buffer, Catapult requires that the SRAM interface be identical to the double buffer interface for the tool to instantiate the SRAMs.** If you do not make them the same, you will not end up with actual SRAMs in your design. We have provided an example SRAM wrapper that has 128-bit words with 512 words in [dnn-accelerator-syn/example_sram_wrappers/sram_512_128.v](#).

Make sure to test your verilog module wrappers. Look within the SramUnit folder to see how to configure mflowgen nodes to run a testbench. Make sure to cover memory operations from every bank, since this is usually where mistakes occur.

Before adding the generated memories to Catapult, you need to modify the .v file that OpenRAM produces, and insert directives before the module definition and after the end-module, like this:

```
// synopsys translate_off
module sram(...)
...
endmodule
// synopsys translate_on
```

We need the Verilog module for the memory in order to simulate the design. However, we do not want the Verilog module to be synthesized. Unfortunately, in the current release of Catapult, the Verilog module for the memory is present in both the RTL used for simulation and the one used for synthesis. Therefore, we need to add these directives so that Design Compiler ignores the module. `// synopsys translate_off` tells Design Compiler to ignore everything that appears after this comment, and `// synopsys translate_on` tells Design Compiler to stop ignoring. You can see an example of how this is used in either of the example SRAM wrapper files.

Now that we have the modified Verilog we can add this as a library to Catapult. Open Catapult in the homework 5 solution folder using `make gui`, and then select “Hierarchy” tab from the left panel. While you are on this tab, open Tools → Memory Generator. As shown in Figure 1, add the Verilog file and check the parse box. Finally, specify an output directory for the Catapult generated files.

Hit Next, and verify that the width and depth of the memory are correct, refer to Figure 2. You want to check the ignore button for all parameters as this will tell Catapult not to change these parameters. Hit Next, and then as shown in Figure 3, specify the port configuration, as well as the pin function for every pin. Make sure that you correctly mark active low pins.

Note: The memory macros that we have generated for you are have 1 read/write port, and one read port. Catapult likes to try and schedule both the reads and writes to the accumulation buffer using the 1 read/write port, so we have to create the wrapper around that memory macro that disables the first read port. We have included this file [dnn-accelerator-syn/example_sram_wrappers/sram_256_32_1r1w.v](#). Configuring this as a 1 read 1 write SRAM is shown in Figure 4.

Hit Next and then Generate. The log should have no errors. Repeat this process for each SRAM macro/wrapper.

We have provided two scripts: `generate_sram_512_128_1rw1r_lib.tcl` and `generate_sram_256_32_1w1r_lib.tcl` that do this whole process for you. These two scripts will generate the memory libraries for the SRAM wrappers in the [dnn-accelerator-syn/example_sram_wrappers/](#) folder. It can be tricky to get the memory builder to generate the library without issues, so feel free to use these scripts as a starting point for any memory wrapper that you would like to generate. Use the command below to run the scripts:

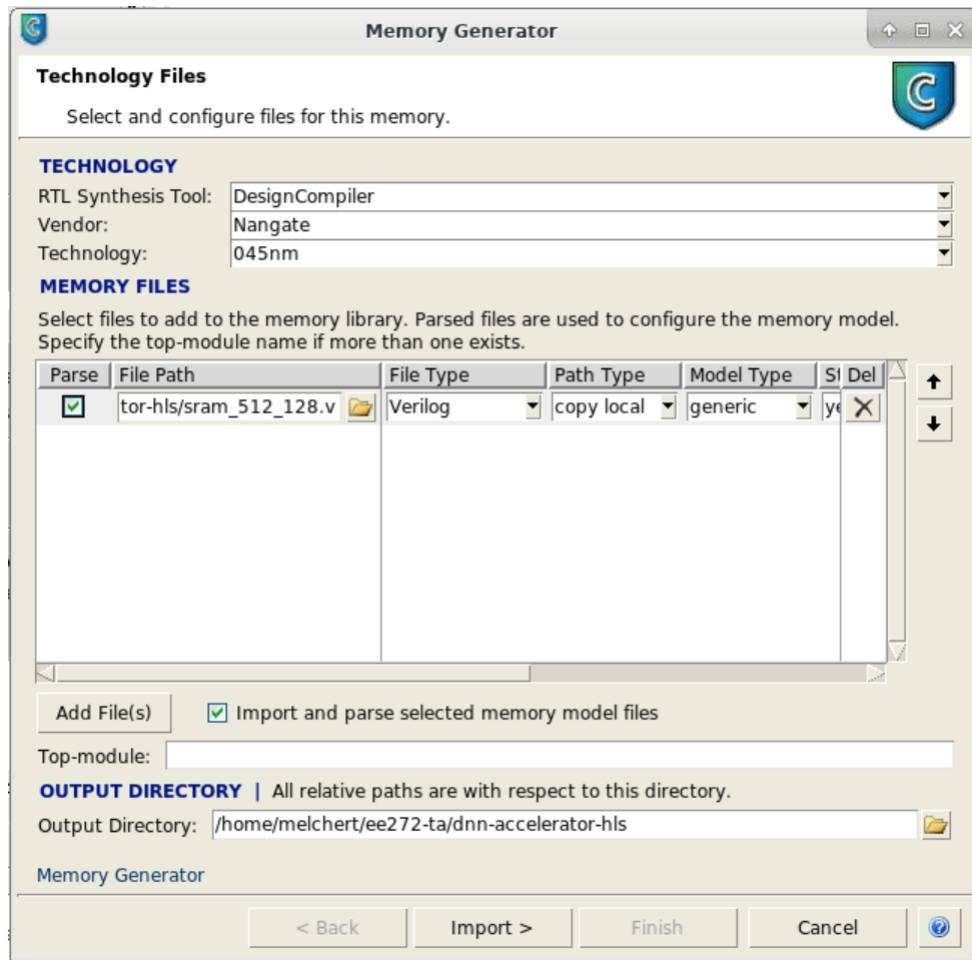


Figure 1: Add the Verilog file, check the parse box, and specify an output directory.

```
catapult -shell -file sram_256_32_1w1r.tcl
catapult -shell -file sram_512_128_1rw1r.tcl
```

Now, go into the Libraries step, click on Search Path, and add the directory you gave to Catapult in the previous step. Copy the command that appears in the transcript window at the bottom into `scripts/common.tcl` so you don't have to manually repeat this step.

```
solution options set ComponentLibs/SearchPath path_to_lib -append
```

You should now see your SRAM listed as a compatible library. As shown in Figure 5, make sure to select your SRAM and unselect Sample RAMs. Hit Apply, and copy the command that appears into `scripts/generic_libraries.tcl`.

```
solution library add sram_512_128_1rw1w
```

Catapult should automatically map arrays to the appropriate memory, but you can also

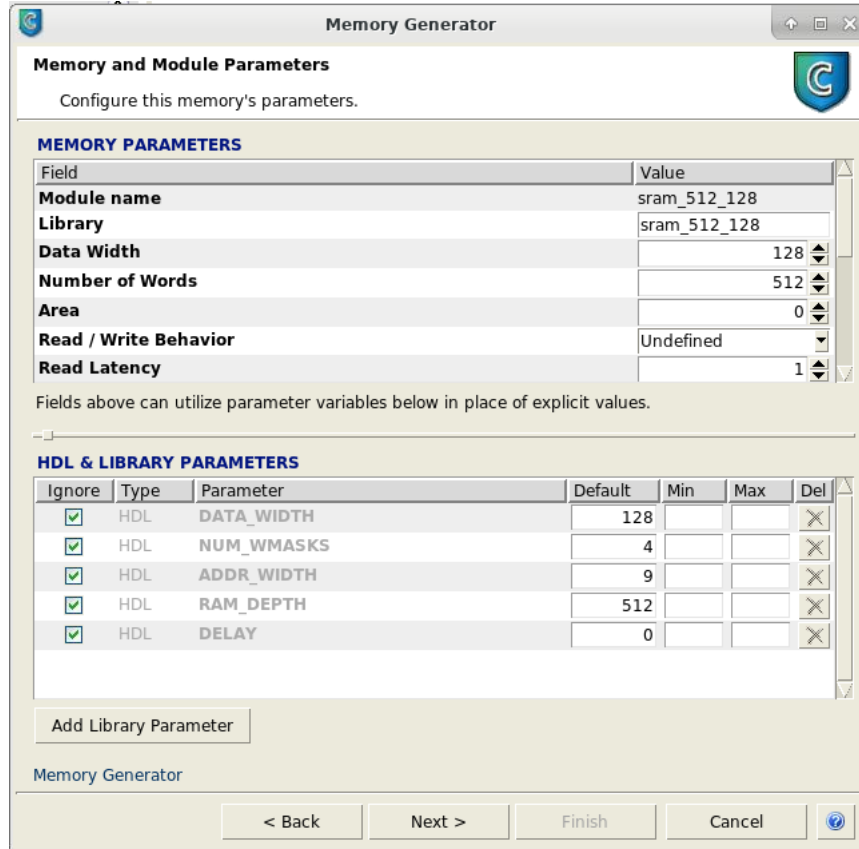


Figure 2: Add the default parameters from the Verilog file.

explicitly specify this with the `MAP_TO_MODULE` directive.

6 Your Task

6.1 Run Pipe Cleaners

- Run both pipe cleaners for GcdUnit and SramUnit to familiarize yourself with the tools.
- After running the flow examine the reports as well: premapped check design, final QOR, timing, area and power.

6.2 Develop CNN Accelerator Synthesis Flow

- Next, develop a new flow in mflowgen for synthesizing your HLS-generated CNN RTL design. Place this flow in the “ConvHLS” folder in the repository mirroring the structure of the two pipe cleaner designs.

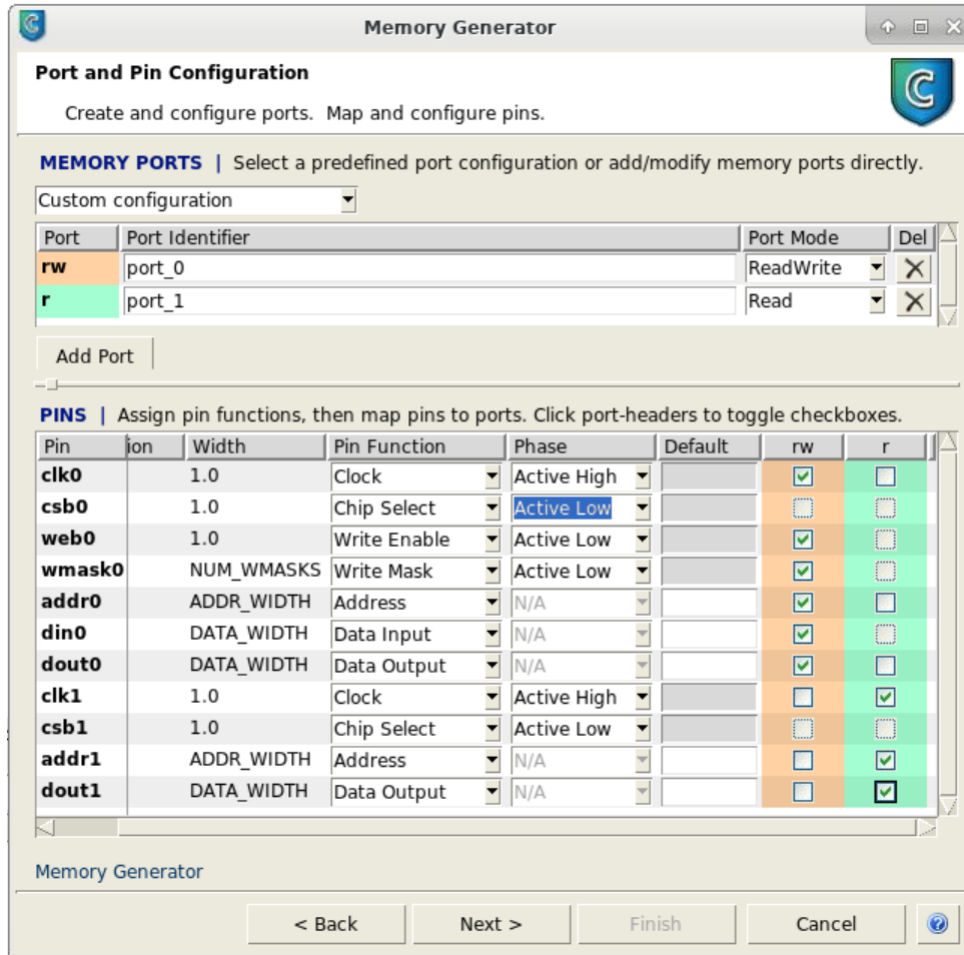


Figure 3: Select the correct port configuration and pin function.

- You can use either pipe cleaner as a starting template.
- Add a new “sram” node to your flow that uses the OpenRAM generated SRAM macros. You can use OpenRAM to generate SRAM .lib/.lef files if you would like (to do that study its READMEs). Alternatively, you can use macros that we have provided for you. Make sure in this step to add any SRAM wrappers that you have created that are instantiated in your HLS design.
- Add a new “rtl” node that runs Catapult using your new memory macros and generates “design.v”. You will need to add some edits to your RTL (which can be done in the configure.yml) so that DC can synthesize your design. For example, you might find several `unreg_hier` modules that are defined more than one time in the generated verilog codes. Synthesis tool will complain if it see duplicated module definition. Remove them and only keep one. Hint: use `sed` and `echo` in your yml file, or you can write a script to do it.
- Double check that `design.v` has the SRAM macros you intended to have.

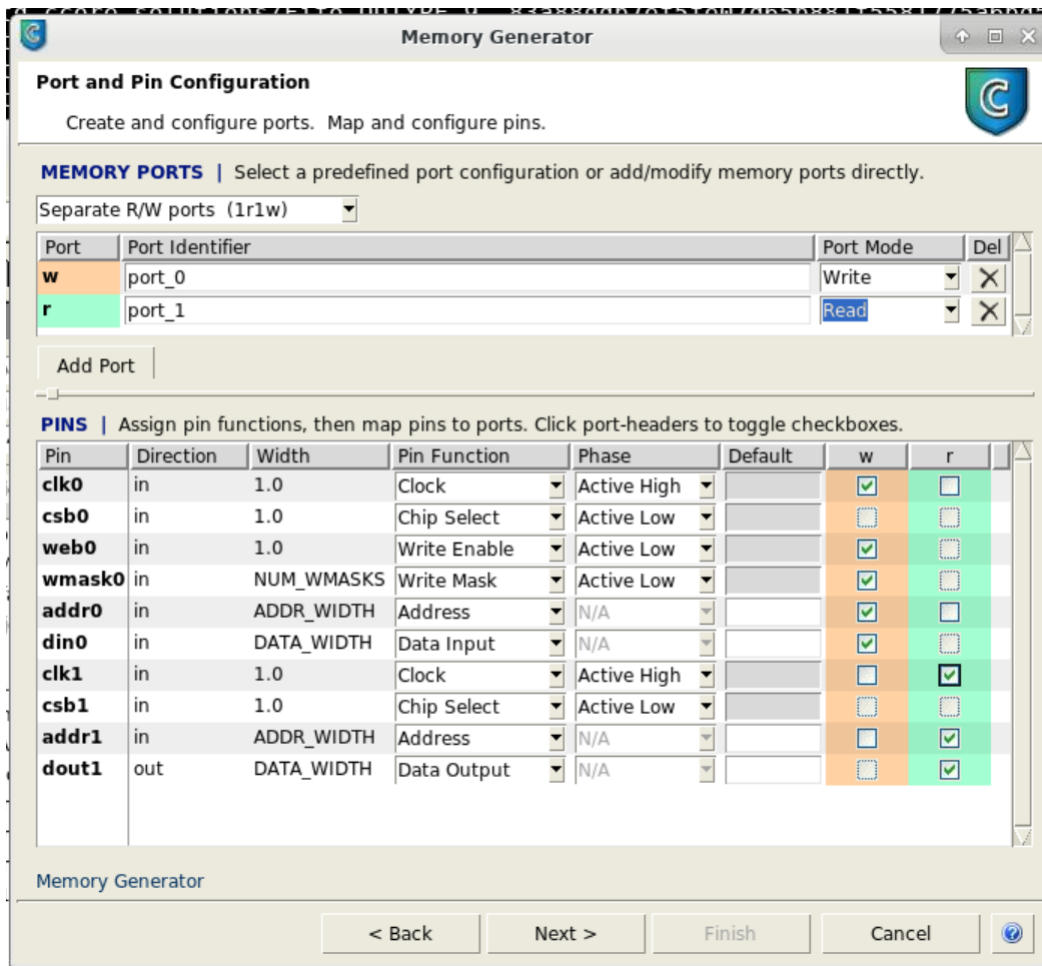


Figure 4: Select the correct port configuration and pin function.

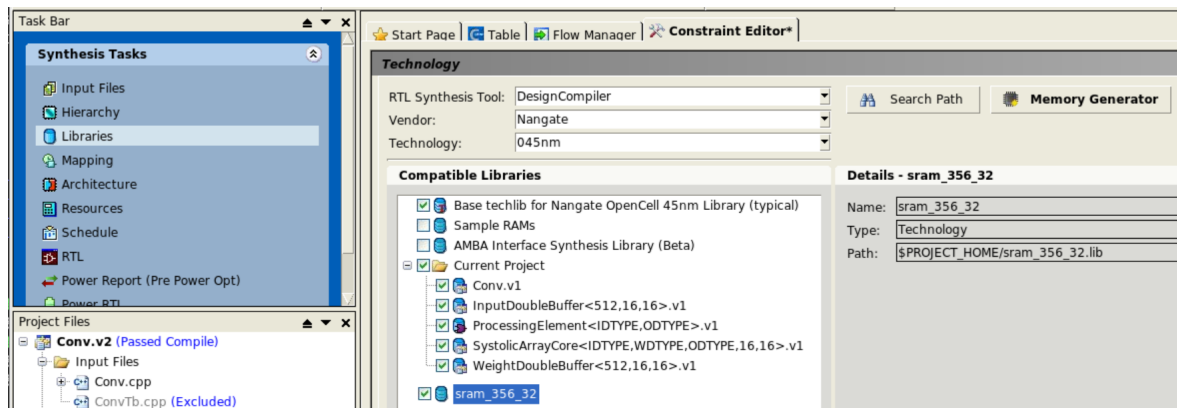


Figure 5: Select your SRAM and unselect Sample RAMs.

- Make the clock period parameterizable for the “rtl” node. Parameters set in `construct.py` can be accessed from scripts within each step by accessing them as an environment variable. Take a look at `$MFLOWGEN_HOME/steps/synopsys-dc-synthesis/configure.yml`

to see how to add a parameter to `configure.yml`. You will then need to modify `scripts/common.tcl` to read the clock period from the environment variable by using `$$:env(clock_period)`.

- Connect the Catapult node to the synthesis node.

6.3 Run CNN Synthesis Flow on HLS Design

- Run synthesis using your new flow. **Make sure there are no errors in any of your log files. Double check that the synthesized netlist has the SRAM macros you intended to have.**
- For accurate power estimation you will need to dump a saif (switching activity file) and connect it to the dc node. There is an example of this in the GcdUnit which uses `vcd2saif` to perform this step. You can use a VCD from any HLS simulation you have run to generate the saif. For HLS, you will need to add vcd dumping in the Conv block in [dnn-accelerator-hls/build/Conv.v1/concat_sim_rtl.v](#), similar to how it is done for RTL. You can test your flow without power and then come back and add this step.
- From the synthesis reports, determine the critical path, area and power of your design. Most of this is available in the QoR report. The critical path will be in the timing report as the first path in each “Path Group” group (e.g., REGIN, REGOUT, ideal.clock).
- Run your design at several frequencies, starting with a slow frequency (for example 10 MHz) to debug your flow, and then increase it till you reach the knee of the area-frequency curve. Hint: You can use mflowgen to make this exploration easier for you: <https://mflowgen.readthedocs.io/en/latest/ug-param-space.html?highlight=sweep#sweeping-large-design-spaces>. For each run, examine the timing report to find the critical path. The slack in the timing report will tell you how much you can shorten the clock period. As you do these runs, build a table (include this in your homework report). If runtime becomes excessively slow and fails timing, do not continue to the next rows and just mark them as not meeting timing. Here are some processors that were done in 130 nm: https://en.wikipedia.org/wiki/130_nm_process, if you follow the links you will see most were able to hit 1GHz performance.
- Try ways to optimize your design to improve performance, power and area. Describe your optimizations and the benefits they produce in your report.

6.4 Run CNN Synthesis Flow on Verilog Design

- Do the same as above, but instead of using your HLS design, use the RTL design from homework 3. You can use the solution version if you want. Remember to instantiate real SRAM macros in your RTL design (currently the homework 3 solution simply has functional models for these). Create a separate graph for your RTL solution and put it

Table 1: HLS synthesis results at different frequencies.

Target clock frequency	Critical path	Total area	Was timing met?
Slow clock – 10 MHz			
50 MHz			
100 MHz			
200 MHz			
250 MHz			
500 MHz			

in “ConvVerilog” folder in the repository mirroring the structure of the two pipe cleaner designs. You may have to tweak the RTL a bit to get it to compile (ex. change top design name). Since the RTL is parameterized you need to make sure that the default parameters of the top level Verilog module reflect the design you want to synthesize. You can confirm this by looking at a waveform of the RTL with the parameters you have selected.

- If you are using the SRAM behavior model from OpenRAM, note that there is a read out delay controlled by a *DELAY* parameter. The default delay value is 3ns and that may cause a problem in simulation. In our ConvVerilog testbench we set the clock period to be 6ns, and the data will be readout 3ns after the negative edge, which will make the registers catch the wrong values. We have changed the pre-generated macro models to 1ns to prevent this issue. If you generates your own ones, you should check this *DELAY* parameter and set it to a reasonable value.
- Once synthesis is complete double check that the synthesized netlist has the SRAM macros you intended to have.
- Try ways to optimize your design to improve performance, power and area. Describe your optimizations and the benefits they produce in your report.
- Compare the results of running synthesis on hand-written Verilog and HLS-generated Verilog designs. Why do you think one was able to hit a higher clock period than the other? Why did one take more area? Explain the differences you observe clearly in your report.

7 Documentation

These tools are complicated and documentation is of tremendous help. For this assignment, the most useful document will likely be the Design Compiler user guide (</cad/synopsys/doc/dcug.pdf>).

Another good way to get help is on the `dc_shell` command line. If you see a command in one of the provided scripts and you’re not sure what it means, open a `dc_shell` by running

Table 2: Verilog synthesis results at different frequencies.

Target clock frequency	Critical path	Total area	Was timing met?
Slow clock – 10 MHz			
50 MHz			
100 MHz			
200 MHz			
250 MHz			
500 MHz			

`dc_shell`. Then enter `*name_of_command* -help`, substituting in the tcl command you want to know about. This will typically give you a basic description of what the command does and what required or optional arguments it accepts.

You can also get help with error/warning messages that pop up during synthesis. Every error/warning is followed by a code, which is a short phrase in all caps followed by a hyphen and a number (LINK-1 is an example). In a `dc_shell`, you can type `man *ERROR CODE*`, and it will give a more detailed explanation of the error or warning and what types of things would cause it to occur.

8 Code Submission and Grading

Please submit the following in a `.tar.gz` file to Canvas:

- Your ConvHLS and ConvVerilog folders.
- Area, timing, and power reports. The timing report must not report any timing violations (all slacks must be positive).

Please write the following in a PDF report and submit it on Canvas using the provided report template:

- Your strategy and approach. Your report should justify the constraints you added, along with any other changes you made to the constraints and synthesis flow.
- Tables 1 and 2 described above.
- Ways in which you tried to optimize your design to improve performance, power and area. Describe your optimizations and the benefits they produced.
- A summary of your design’s final area, frequency and power.

- Comparison of hand-written Verilog and HLS-generated Verilog designs, along with an explanation for the observed differences.

Name the .tag.gz and PDF files using your and your partner's full names. The grading of this assignment will be based on whether or not you were able to successfully synthesize your designs, the constraints you added, and the quality of your report.